五、打包解包

詹文翰 高级实验师
zhanwenhan@163.com

课程安排


1
需求分析
2
系统设计
3
程序框架
4
软件测试
5
打包解包
6
压缩解压
7
加密解密
8
图形界面
9
软件发布
10
项目演示

实现功能

文件类型支持：支持特定文件系统的特殊文件（管道/链接/设备等）

元数据支持：支持特定文件系统的文件元数据（属主/时间/权限等）

自定义备份：支持筛选需要备份的文件（路径/类型/名字/时间/尺寸/用户）

打包解包：将所有备份文件拼接为一个大文件保存，支持多种类型的打包解包算法

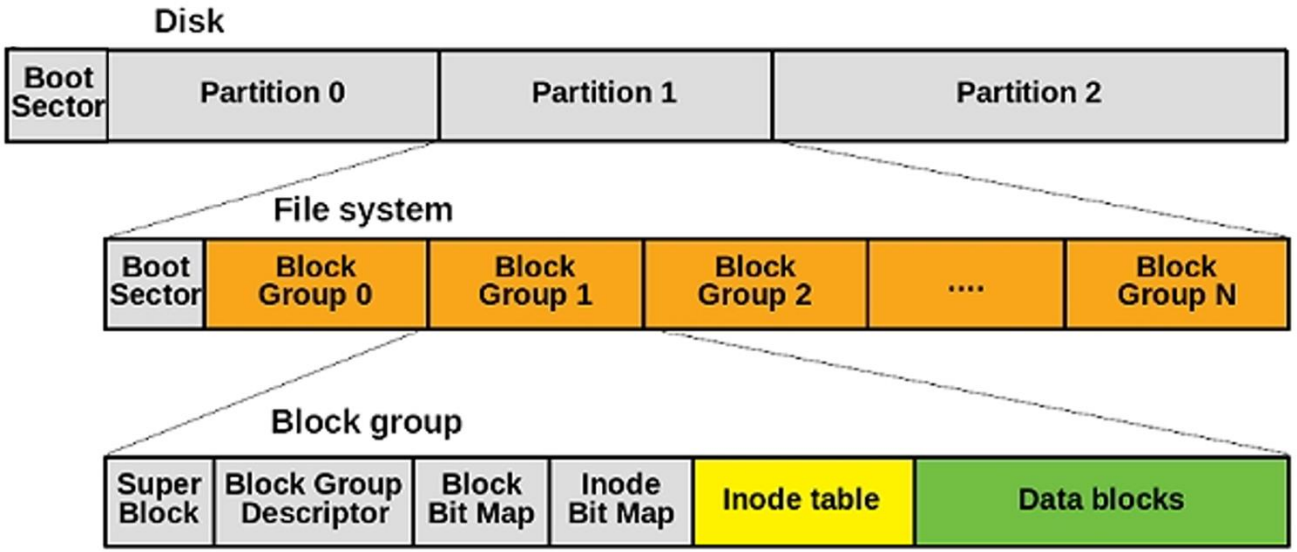
文件系统

文件系统原理

文件操作方法

Unix文件系统

典型 Unix 磁盘布局（ext2 文件系统）



ext2 官方文档：https://www.nongnu.org/ext2-doc/ext2.html

Unix文件系统

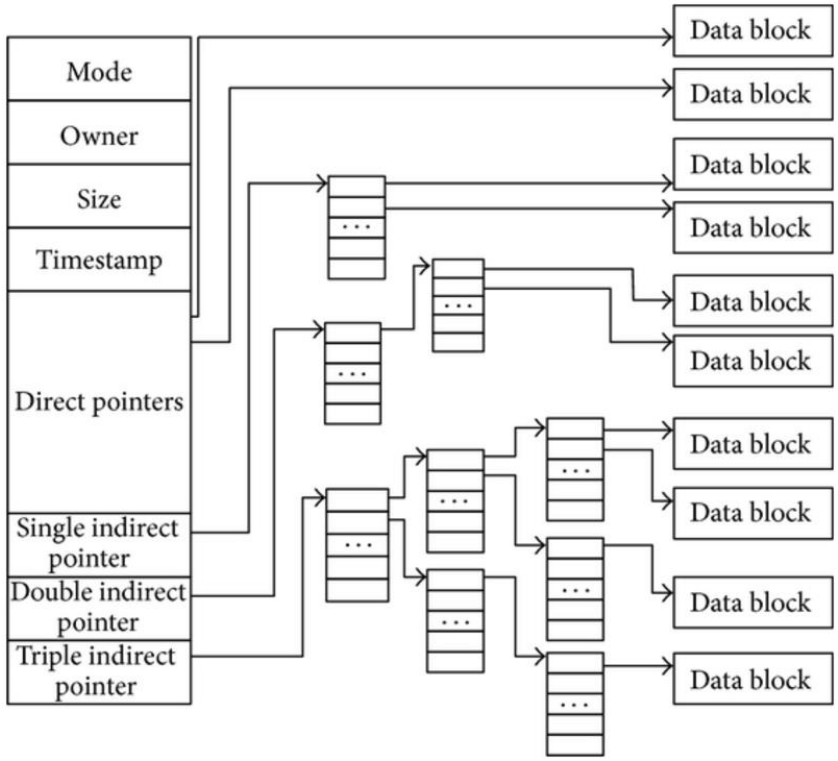
Unix系统中的文件类型：

- 普通文件 
- 目录文件 
- 字符设备文件 
- 块设备文件 
- 套接字文件
- 管道文件 
- 链接文件 

Unix文件系统

普通文件的实现

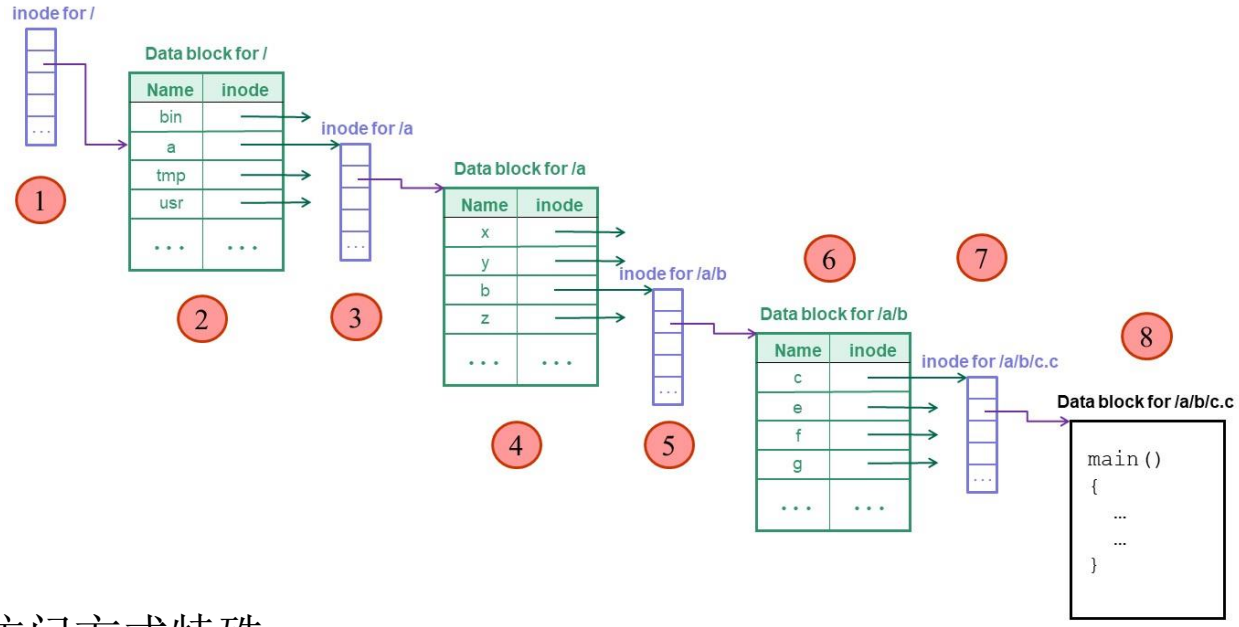
- UNIX 普通文件采用索引结构
- 索引数据结构——inode
- inode存储了文件的所有元数据
- 特殊的索引方式：
  - 直接索引　　10个
  - 一级间接索引　　1个
  - 二级间接索引　　1个
  - 三级间接索引　　1个



Unix文件系统

目录文件的实现

- 目录也是文件！
  - 有特殊结构的文件
  - 以访问 /a/b/c.c 为例

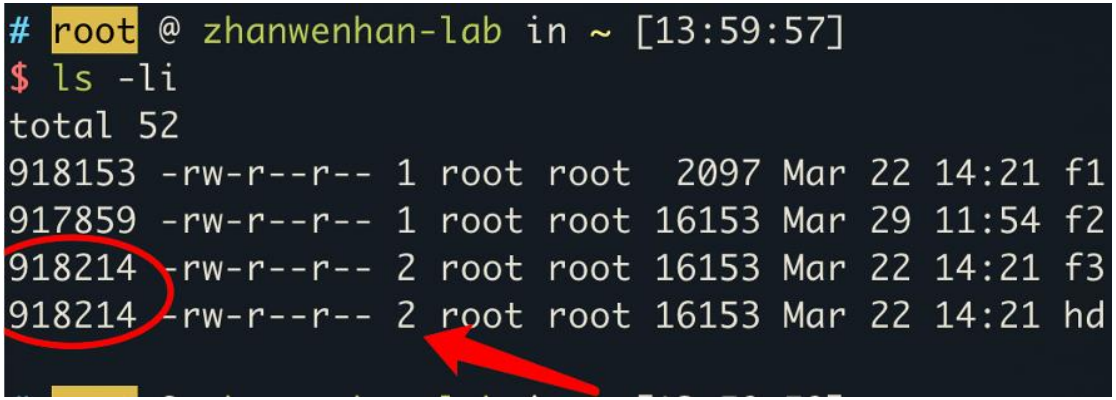


- 访问方式特殊
  - 自动维护目录结构（.和..）
  - 保护，防止误操作

Unix文件系统

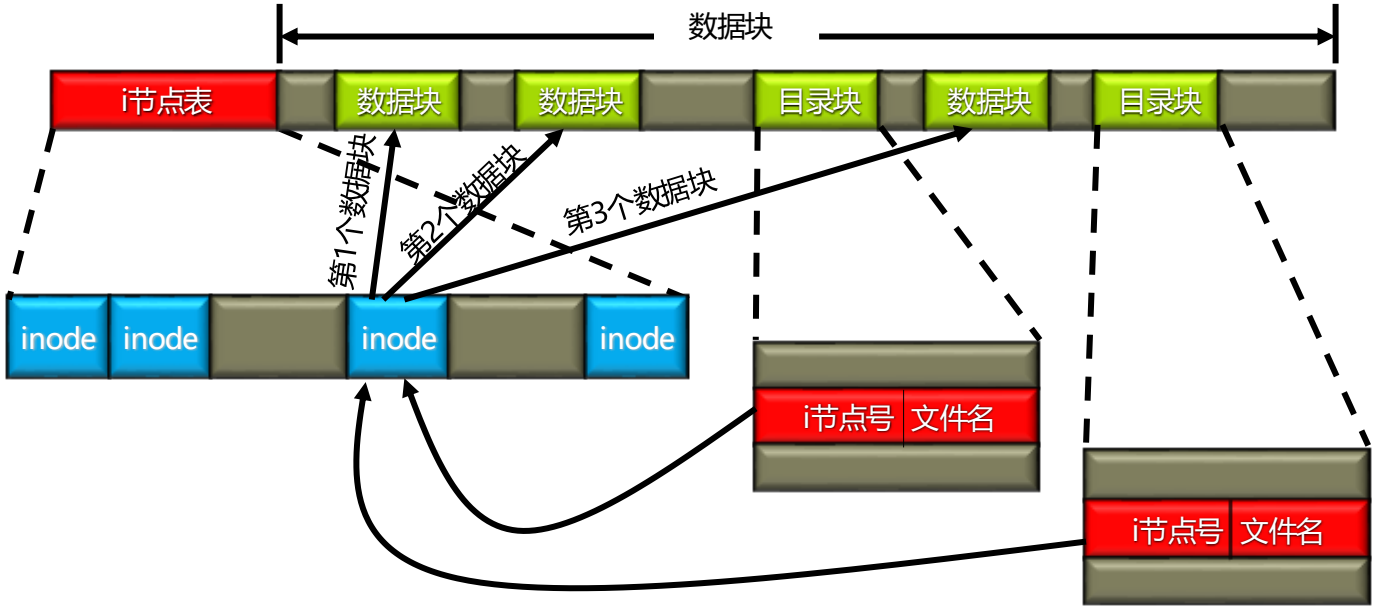
硬链接

- 指向同一个 inode 的目录项的数量
- 硬链接实现的是 inode 共享
- ./ 和 ../ 会影响到链接数



Unix文件系统

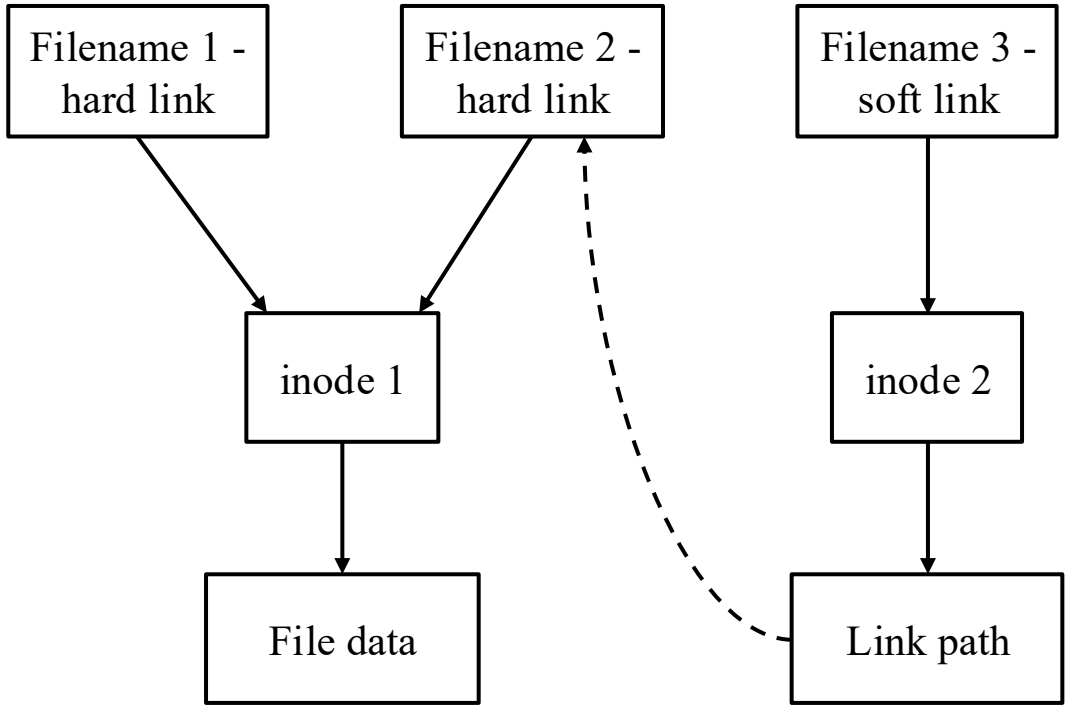
例：指向同一个 inode 的两个目录项。



Unix文件系统

软链接

- 软链接本身就是一个文件，有自己的 inode 和数据块。
- 软链接的数据块中存储了另一个文件或目录的路径，而不是文件或目录的实际数据。



Unix文件系统

项目涉及到的系统API（部分）：

open
close
read
write
stat
chown
chmod
utime
opendir
closedir
readdir
rewinddir
mkdir
mkfifo
unlink
link
symlink
……

Unix文件系统

项目涉及到的系统API（部分）：

open
close
read
write
stat
chown
chmod
utime
opendir
closedir
readdir
rewinddir
mkdir
mkfifo
unlink
link
symlink
……

man手册

文件遍历&打包解包

打包解包原理

文件遍历方法

打包解包

打包：将多个文件/目录整合为一个大文件的过程。

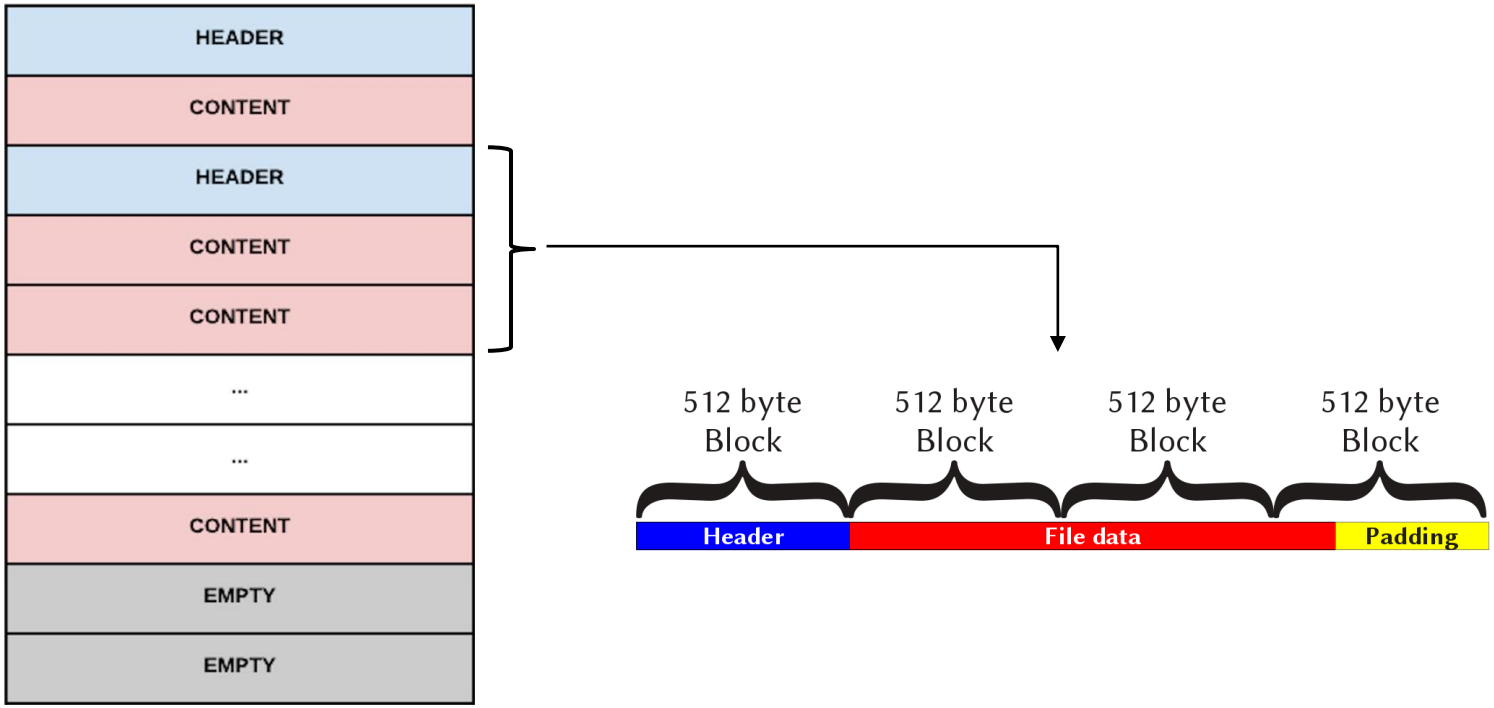
解包：将多个文件/目录从一个大文件中按原样恢复的过程。

大部分压缩软件均支持同时对文件打包。最经典的文件打包程序：Tar。

Tar打包原理

Tar是Unix和类Unix系统上的归档打包工具，可以将多个文件合并为一个文件，打包后的文件名亦为“tar”。目前，tar文件格式已经成为POSIX标准。

Tar归档文件由一系列文件对象通过线性排列的方式组合而成。每个文件对象都包含一个或者若干个512字节的记录块，并以一个头记录开头。文件数据按原样写入，但其长度舍入为512字节的倍数。



头记录数据结构

```
union Record
{
    union
    {
        // Pre-POSIX.1-1988 format
        struct
        {
            char name[100];      // file name
            char mode[8];        // permissions
            char uid[8];         // user id (octal)
            char gid[8];         // group id (octal)
            char size[12];       // size (octal)
            char mtime[12];      // modification time (octal)
            char check[8];       // sum of unsigned characters in block (octal)
            char link;           // link indicator
            char link_name[100]; // name of linked file
        };

        // UStar format (POSIX IEEE P1003.1)
        struct
        {
            char old[156];      // first 156 octets of Pre-POSIX.1-1988 format
            char type;                  // file type
            char also_link_name[100]; // name of linked file
            char ustar[8];              // ustar\000
            char owner[32];             // user name (string)
            char group[32];             // group name (string)
            char major[8];              // device major number
            char minor[8];              // device minor number
            char prefix[155];
        };
    };

    char block[512]; // raw memory (padded to 1 block)
};
```

```
#define REGULAR          0
#define NORMAL          '0'
#define HARDLINK        '1'
#define SYMLINK         '2'
#define CHAR            '3'
#define BLOCK           '4'
#define DIRECTORY       '5'
#define FIFO            '6'
```

实现思路

- 将**目录遍历算法**和**打包算法**分开实现和执行
  - 目录遍历算法：
    - 输入：某个目录
    - 输出：目录下的所有文件（全部类型）列表
  - 打包算法：
    - 输入：文件列表
    - 输出：打包文件
  - 解包算法：
    - 输入：打包文件，解包目录
    - 输出：解包数据

实现思路

- 将目录遍历算法和打包算法分开实现的优点和缺点
  - 优点：
    - 便于逻辑解耦
    - 易于实现
    - 便于实现复杂的文件过滤逻辑
  - 缺点：
    - 时间效率和空间效率下降
    - 打包过程中的数据一致性可能会出现问题
  - 目录遍历算法输出文件列表的顺序对解包算法很重要
    - 结论：先序遍历

实现思路

- 实现**目录遍历算法**应该注意哪些问题
  - 硬连接如何处理？
    - 直接复制 or 保持链接关系？
  - 软连接如何处理？
    - 直接复制 or 仅复制链接？
    - 目录软链接可能导致遍历死循环（lstat）
  - 相对路径 or 绝对路径？
    - 建议将程序的当前路径调整为对应目录再开始遍历
  - 如何进行数据过滤？
    - 在遍历过程中对数据进行筛选
    - 仅保存和继续遍历通过筛选的数据

实现思路

- 实现**打包算法**应该注意哪些问题
  - 输入安全性检查
    - 源路径存在性，可访问性
    - 目的路径存在性，可访问性
  - 程序当前路径要和遍历时的当前路径一致
  - 文件类型的支持
    - 普通文件
    - 目录文件
    - 软链接文件
    - 管道文件
    - 设备文件
    - 套接字
  - 文件名超长处理
    - 使用多个header块（如何标识？）

实现思路

- 实现**解包算法**应该注意哪些问题
  - 输入安全性检查
    - 源文件存在性，可访问性
    - 源文件头校验
    - 目的路径存在性，可访问性
    - 目的路径目录为空！
  - 文件恢复
    - 目录文件
    - 软链接文件
    - 管道文件
    - 设备文件
    - 套接字
    - 普通文件（包括数据恢复，注意文件大小）
  - 元数据的恢复
  - 文件名超长处理
    - 使用多个header块（如何确认）

实验环节

完成以下实验内容：

- 完成本课程项目的目录遍历功能
- 完成本课程项目的文件筛选功能
- 完成本课程项目的打包解包功能
- 采用测试驱动的开发方式（TDD）
  - 先编写测试用例
  - 再完成业务代码
  - 保证所有测试用例通过